INFORME DE TALLER PRÁCTICO

Terraform + Docker

Automatizando una arquitectura Node.js + React + Nginx + MySQL, localmente con Docker

---

Jairo Miguel Lara Serrano

Maestría en Ingeniería de Software
Desarrollo de Aplicaciones Empresariales

28 de agosto de 2026

Índice

1. Objetivo del taller

El taller consiste en desplegar, con Terraform y sin ningún proveedor de nube, una arquitectura completa de cuatro capas sobre contenedores Docker locales, y usarla como banco de pruebas para practicar el archivo de estado (`terraform.tfstate`).

Los objetivos concretos son cinco:

1. Desplegar la arquitectura Nginx (balanceador) → React (frontend) → Node.js (backend, con réplicas) → MySQL, usando el proveedor `docker` de Terraform.
2. Ejecutar el flujo completo `init` → `plan` → `apply` → `destroy`.
3. Practicar de forma guiada el archivo de estado: inspeccionarlo, provocar y detectar *drift*, reemplazar recursos puntuales, renombrar direcciones sin destruir infraestructura e importar un recurso creado a mano.
4. Escalar el backend cambiando una sola variable y observar cómo Terraform recalcula el plan y regenera la configuración de Nginx.
5. Revisar qué haría falta antes de llevar este mismo patrón a producción.

Todo corre en local: no se usó AWS, GCP ni Azure en ningún momento, y el taller no consumió crédito de nube.

2. Arquitectura desplegada

Nginx es el único servicio con un puerto publicado al host (`8080`). Reparte `/api/*` entre las réplicas del backend mediante balanceo *round-robin* y sirve `/` haciendo proxy al contenedor del frontend. Todo lo demás vive dentro de la red Docker privada `iac-workshop-net`, sin exposición al exterior.

Tabla 1: Componentes de la arquitectura

Componente	Función	Puerto
Nginx	Balanceador y *reverse proxy*; único punto de entrada	8080 (host)
React (frontend)	Interfaz servida como estáticos desde su propio Nginx interno	80 (interno)
Node.js/Express	API con `/api/health` y `/api/visits`; N réplicas	3000 (interno)
MySQL 8.0	Base de datos con volumen nombrado para persistencia	3306 (interno)

Dos detalles del diseño que resultan importantes más adelante:

- El backend reporta su `hostname` en `/api/health`. Como Terraform fija ese hostname explícitamente (`backend-1`, `backend-2`...), el balanceo de carga se puede comprobar visualmente desde el navegador.
- La configuración de Nginx no es un fichero estático: se genera en el momento del `apply` con `templatefile()` a partir de la lista de réplicas. Cambiar el número de réplicas cambia ese contenido y obliga a recrear el contenedor de Nginx.

3. Entorno y estructura del proyecto

Tabla 2: Herramientas utilizadas

Herramienta	Versión mínima	Verificación
Docker Desktop / Engine	24.x	`docker version`
Terraform CLI	1.5.0	`terraform -version`
Node.js (opcional)	20.x	`node -v`

El proveedor `kreuzwerker/docker` habla directamente con el socket local de Docker, el mismo que usa el comando `docker`. Por eso no hacen falta credenciales de ningún tipo. El único acceso a internet que se necesita es el de la primera ejecución, para descargar el plugin del proveedor y las imágenes base.

Solo el directorio `terraform/` contiene código de infraestructura; `backend/`, `frontend/`, `nginx/` y `mysql/` son las aplicaciones y ficheros de configuración que Terraform construye y despliega.

```
workshop/
  backend/            API Express (health + visits)
  frontend/           UI React que consume la API
  nginx/              plantilla nginx.conf.tpl que Terraform renderiza
  mysql/              esquema inicial de la base de datos
  terraform/
    versions.tf       provider + version de Terraform
    variables.tf      replicas, puerto, credenciales
    locals.tf         lista de alias backend-1, backend-2, ...
    network.tf        red privada iac-workshop-net
    mysql.tf          imagen + volumen + contenedor
    backend.tf        usa "count" para las replicas
    frontend.tf       una sola replica
    nginx.tf          usa templatefile() + upload
    outputs.tf        URLs y nombres de contenedores
```

4. Ciclo básico: de `init` a la aplicación funcionando

Todos los comandos se ejecutan dentro de `terraform/`.

4.1 Paso 1 — Inicializar

```
cd workshop/terraform
terraform init
```

Terraform descargó el plugin `kreuzwerker/docker` v4.5.0, preparó el directorio `.terraform/` y creó el fichero de bloqueo `.terraform.lock.hcl`, que fija esa versión exacta para futuras ejecuciones.

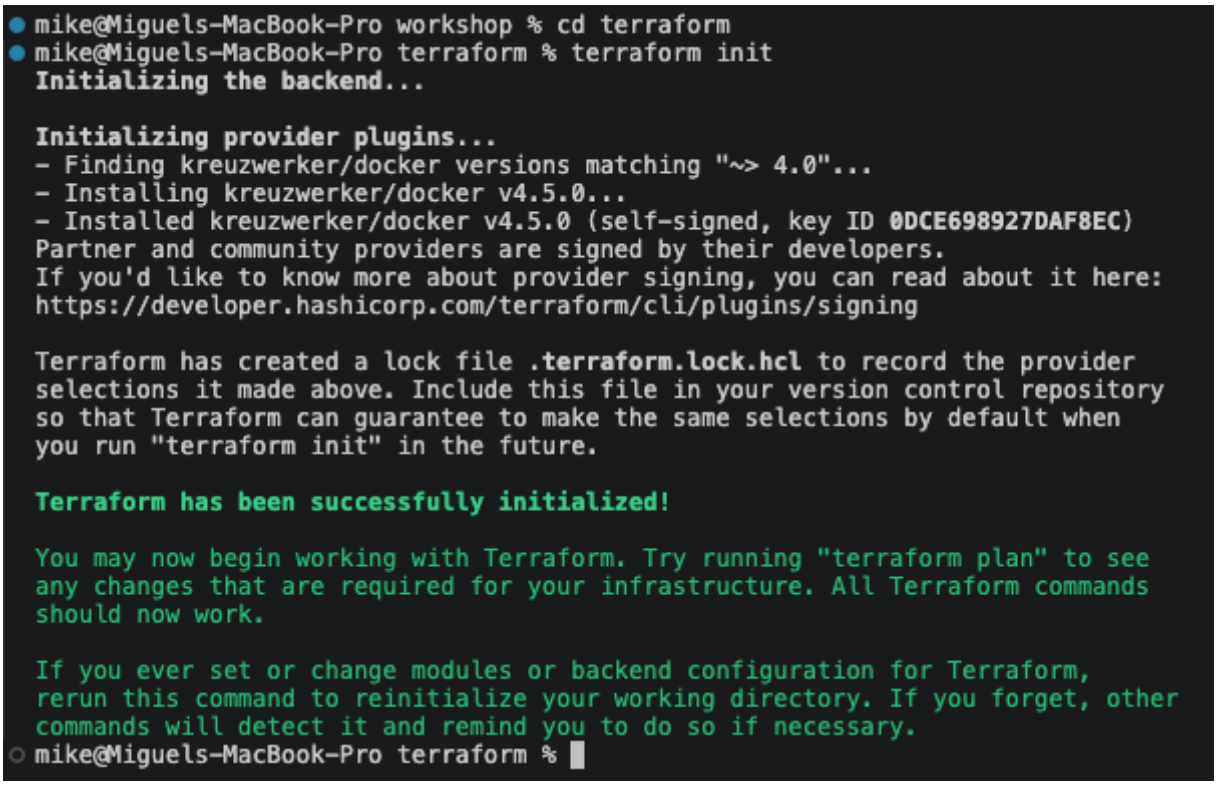

Figura 1: `terraform init`: descarga del proveedor y creación del fichero de bloqueo de versiones.

4.2 Paso 2 — Formatear y validar

```
terraform fmt
terraform validate
```

`fmt` normaliza el estilo del código y `validate` revisa la sintaxis y las referencias internas. Ninguno de los dos necesita que Docker esté corriendo, así que sirven como comprobación barata antes de tocar nada real.

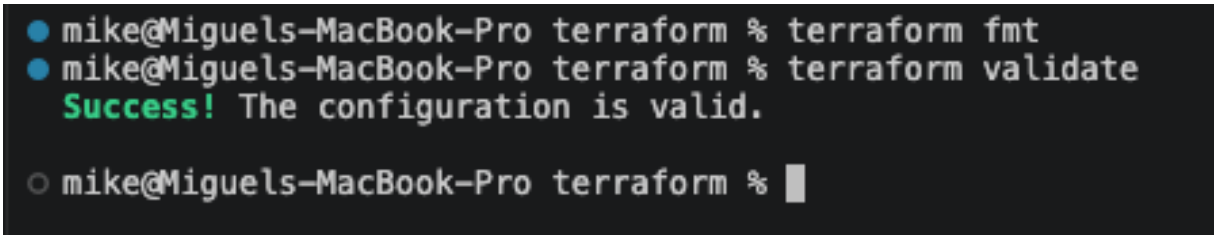

Figura 2: La configuración es sintácticamente válida.

4.3 Paso 3 — Planificar

```
terraform plan
```

El plan anunció **11 recursos a crear** y ningún cambio ni destrucción: la red, el volumen de MySQL, cuatro imágenes y cinco contenedores. Todavía no se creó nada; este es el momento de leer con calma qué va a pasar.

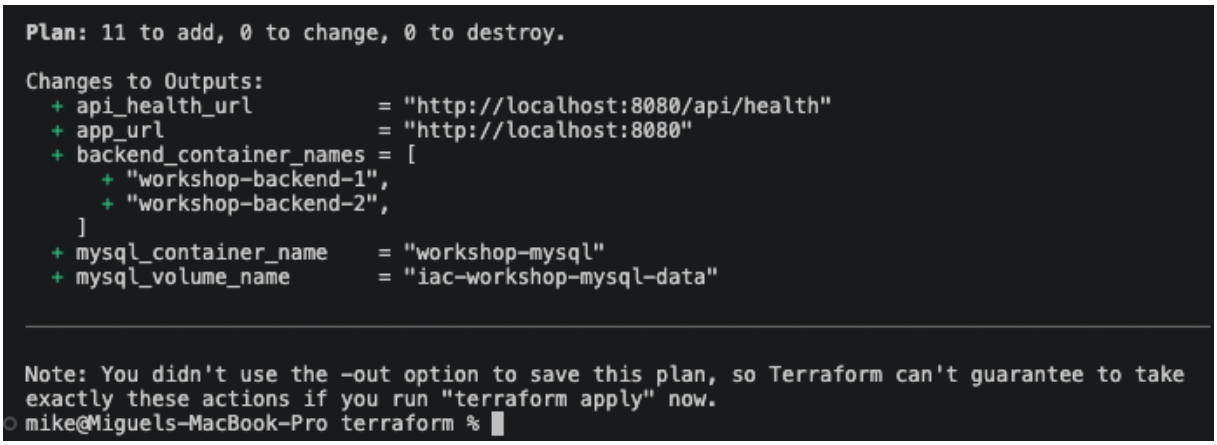

Figura 3: Resumen del plan: `11 to add, 0 to change, 0 to destroy`, junto con los valores que tomarán los *outputs*.

4.4 Paso 4 — Aplicar

```
terraform apply
```

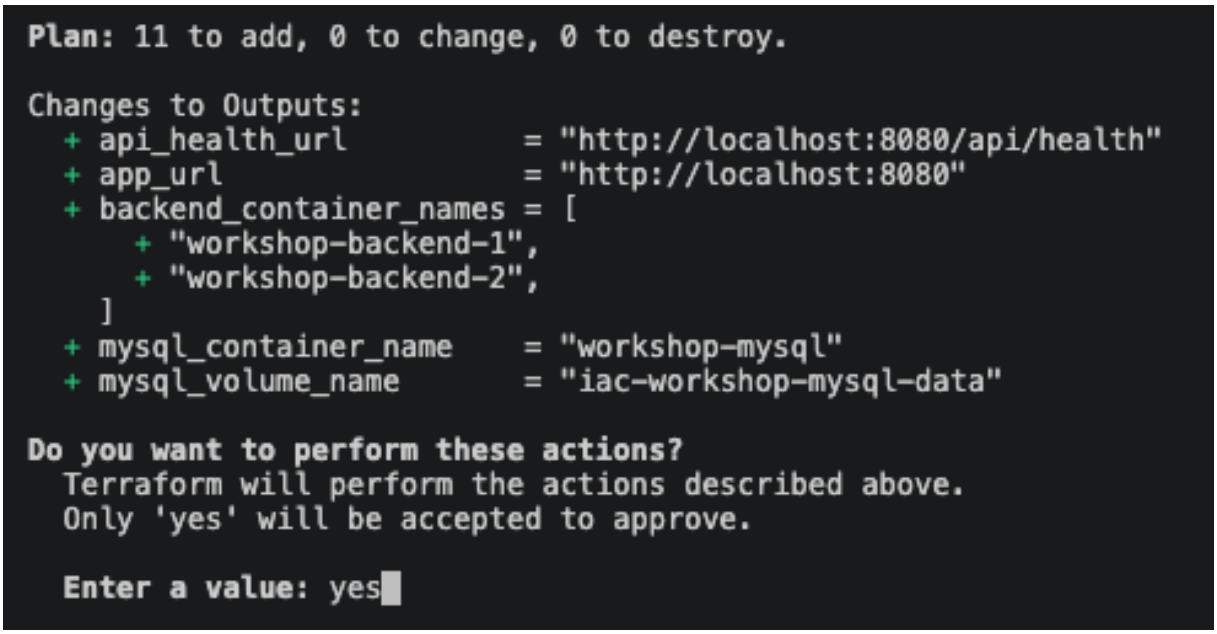

Figura 4: Terraform pide confirmación explícita antes de ejecutar el plan; solo se acepta `yes`.

Durante el `apply` se nota que MySQL tarda alrededor de un minuto en completarse: no basta con que el proceso arranque, porque el recurso declara un `healthcheck` con `wait = true` y Terraform no lo da por aplicado hasta que MySQL responde *healthy*. El `depends_on` del backend obliga a esperar a que eso ocurra.

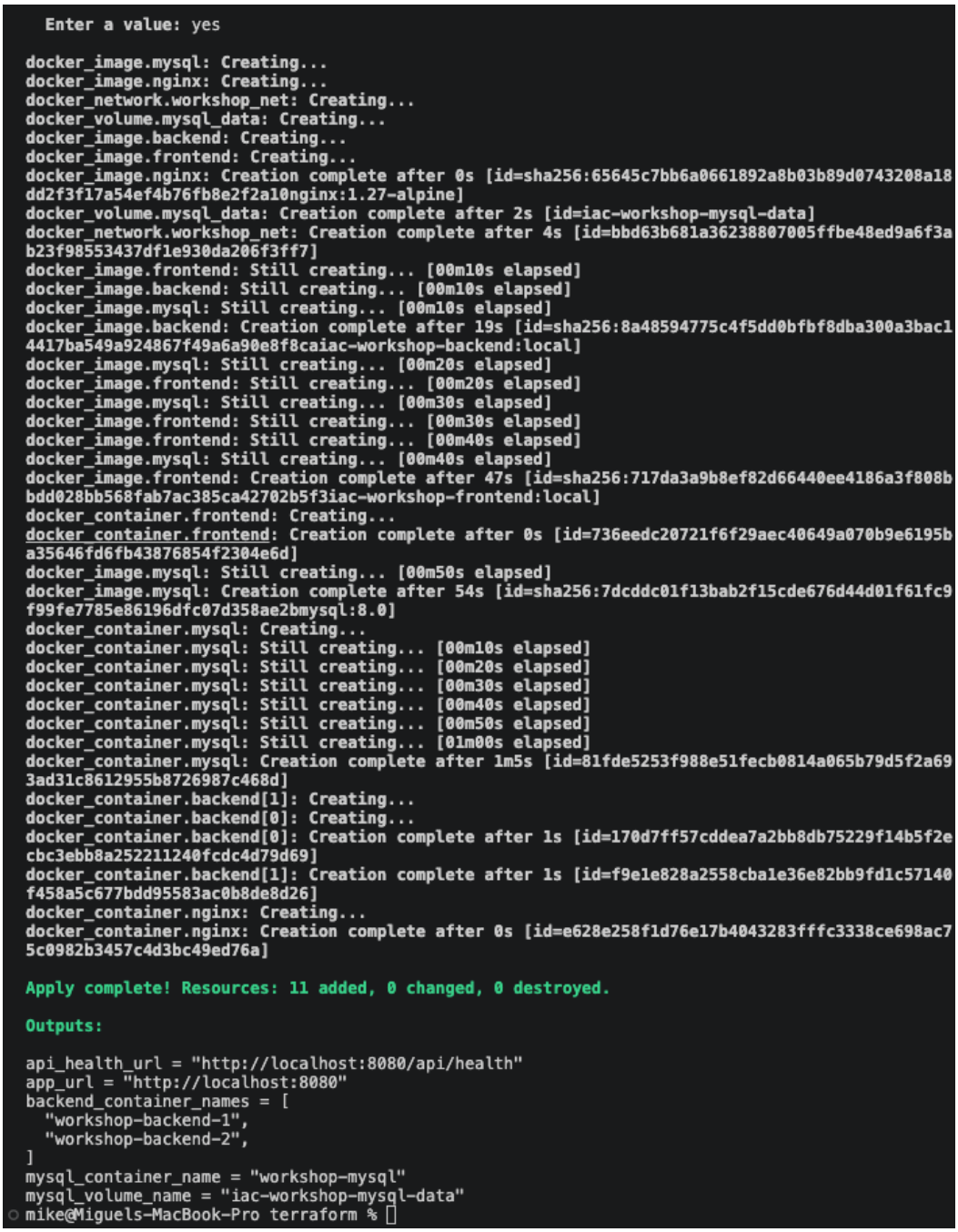

Figura 5: Creación de los 11 recursos en orden de dependencias. MySQL tarda 1m5s por el *healthcheck*; los backends se crean después, en paralelo.

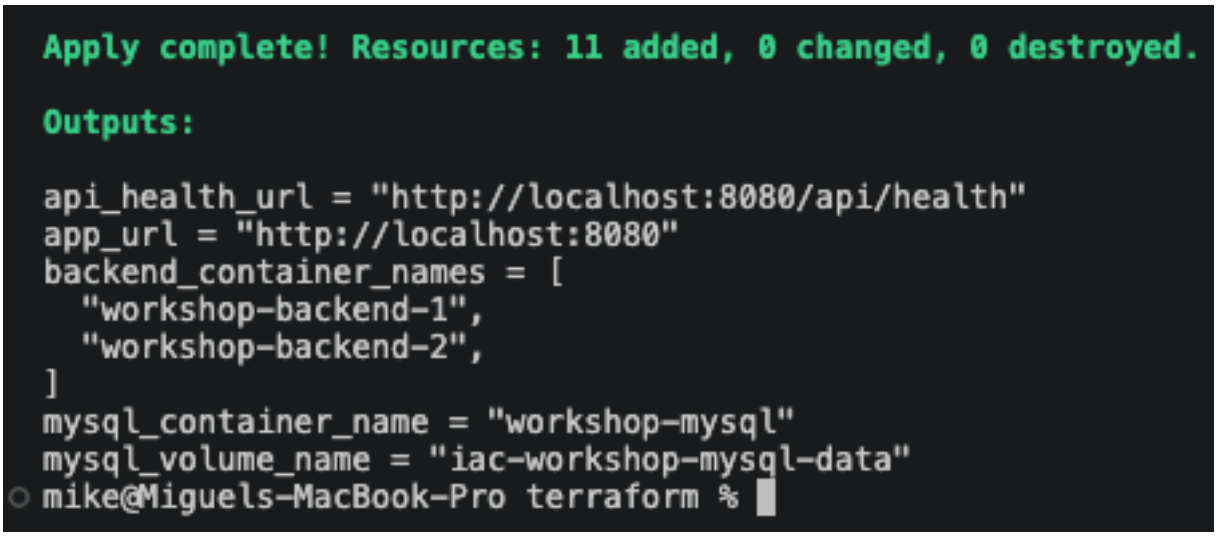

Figura 6: *Outputs* finales: URL de la aplicación, URL de salud de la API y nombres de los contenedores generados.

4.5 Paso 5 — Verificar

```
docker ps
curl http://localhost:8080/api/health
curl http://localhost:8080/api/visits
```

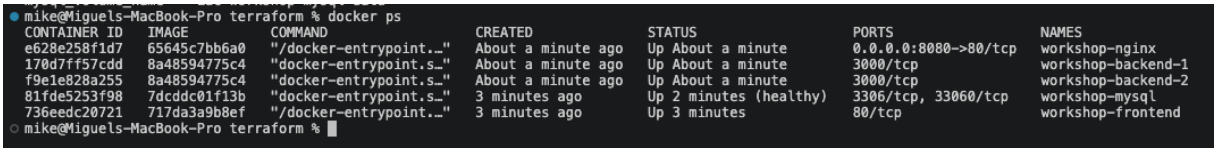

Figura 7: Los cinco contenedores en ejecución. Solo `workshop-nginx` publica un puerto al host (`0.0.0.0:8080->80/tcp`); MySQL aparece como *healthy*.

Abriendo `http://localhost:8080` se ve la interfaz de React leyendo el contador de visitas desde MySQL. El botón que dispara seis peticiones a `/api/health` muestra el balanceo en vivo: las respuestas se reparten entre `backend-1` y `backend-2`.

Figura 8: Aplicación funcionando con dos réplicas. Las seis peticiones alternan entre `backend-2` y `backend-1`.

5. El archivo de estado

Esta es la parte central del taller y la que no tiene equivalente en Docker Compose. Compose puede levantar los mismos servicios, pero no mantiene un registro persistente y consultable de qué es cada recurso, con qué atributos y cómo se relaciona con los demás.

5.1 Inspeccionar el estado

```
terraform state list
```

Lista las direcciones de todos los recursos administrados, incluyendo el índice de cada réplica del backend.

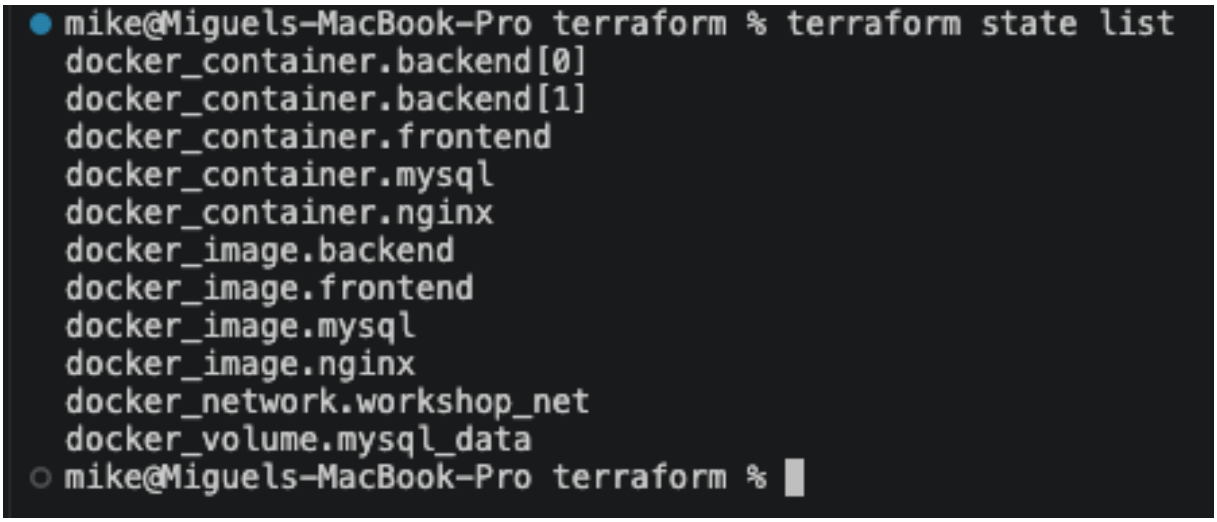

Figura 9: Las once direcciones del estado. Nótese `docker_container.backend[0]` y `docker_container.backend[1]`: `count` produce dos recursos independientes en el grafo.

```
terraform state show "docker_container.backend[0]"
```

Figura 10: Todos los atributos conocidos de una réplica concreta: su ID real de Docker, su IP interna (`172.21.0.4`), su hostname (`backend-1`), su alias de red y su usuario no root (`node`). Las variables de entorno aparecen ocultas por estar marcadas como sensibles.

```
terraform show
```

Figura 11: Volcado completo del estado en formato legible: imágenes, red con su subred `172.21.0.0/16`, volumen con su punto de montaje real en el host, y los *outputs*.

5.2 Idempotencia

```
terraform plan
```

Sin haber tocado nada, Terraform re-consulta la infraestructura real, compara contra el estado y concluye que no hay nada que hacer. Esto es la idempotencia declarativa en acción.

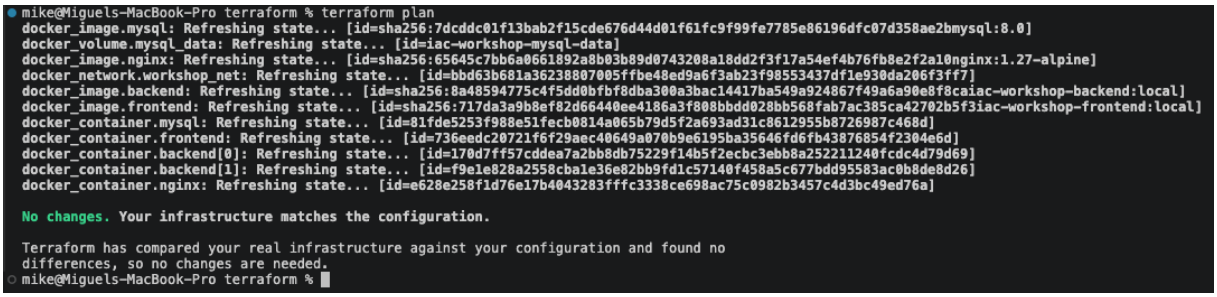

Figura 12: `No changes. Your infrastructure matches the configuration.`

5.3 Provocar y detectar *drift*

El *drift* es la desviación entre la infraestructura real y lo que el código declara, típicamente por un cambio manual. Se simuló borrando un contenedor directamente con el CLI de Docker, sin pasar por Terraform:

```
docker rm -f workshop-backend-2
terraform plan
```

Terraform detectó que `docker_container.backend[1]` ya no existe en la realidad aunque el estado dice que debería, y propuso recrearlo.

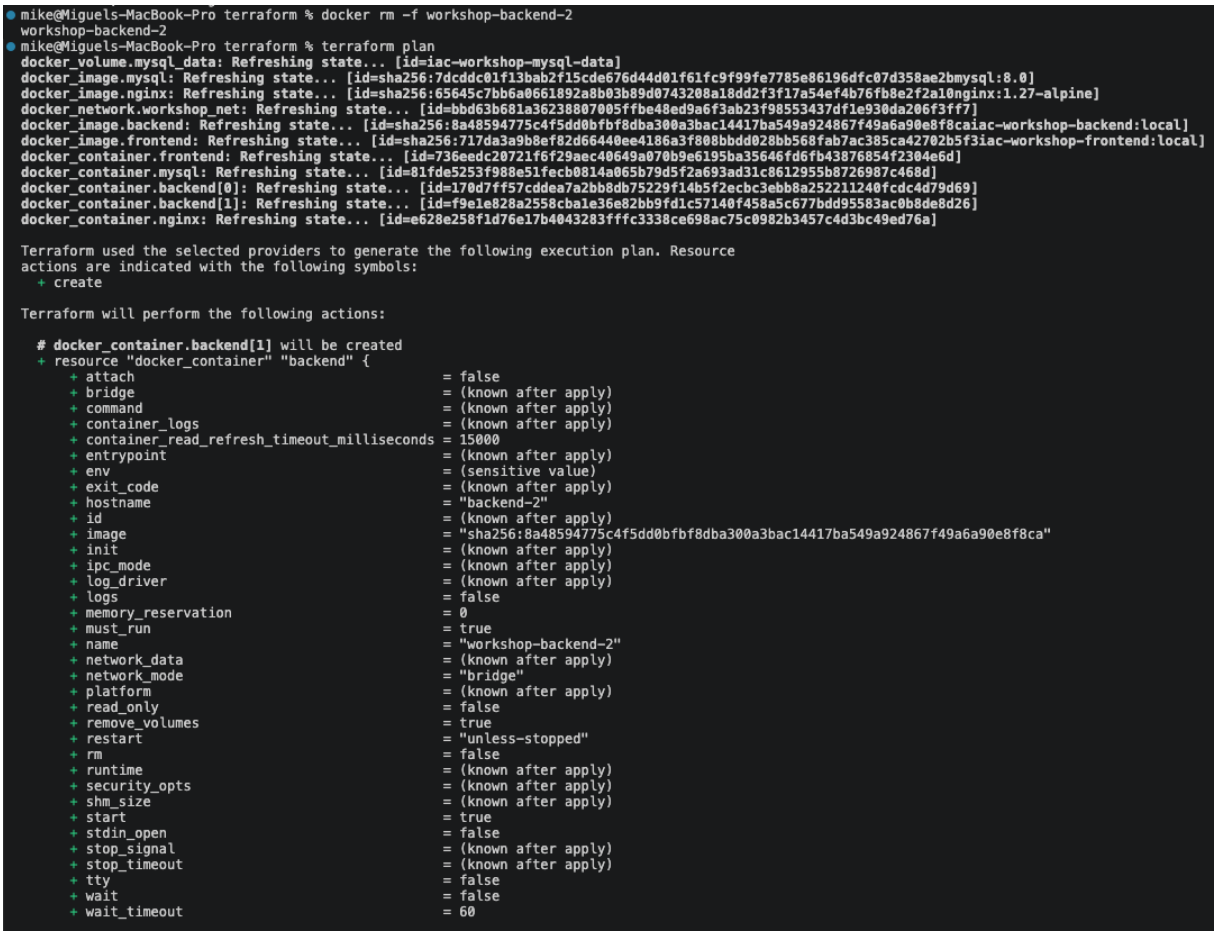

Figura 13: Detección del *drift*: el recurso desaparecido se marca para creación, con `hostname = "backend-2"` y su alias de red.

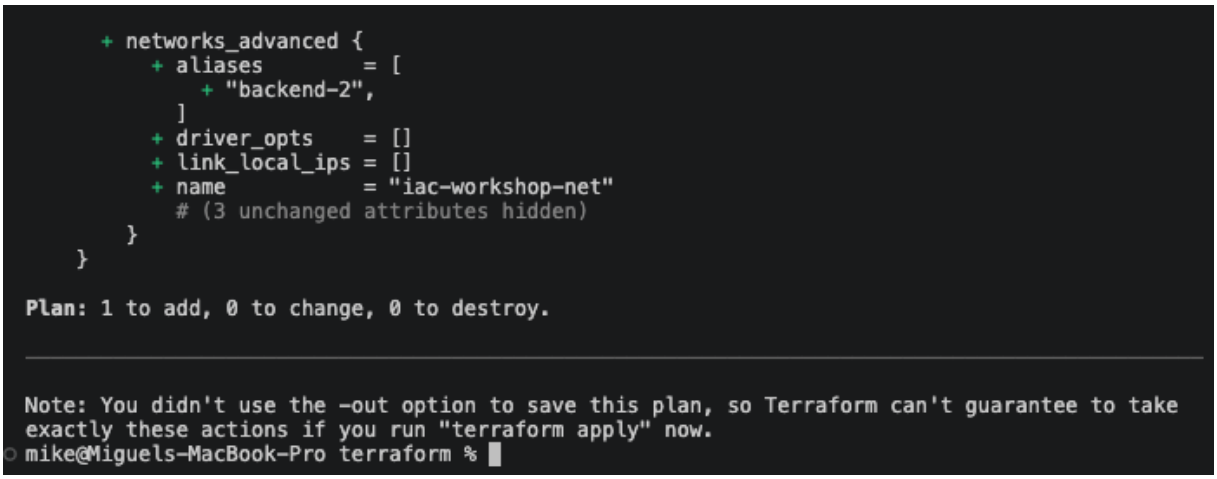

Figura 14: Resumen: `1 to add, 0 to change, 0 to destroy`. Terraform propone tocar solo el recurso afectado.

5.4 Corregir el *drift*

```
terraform apply
```

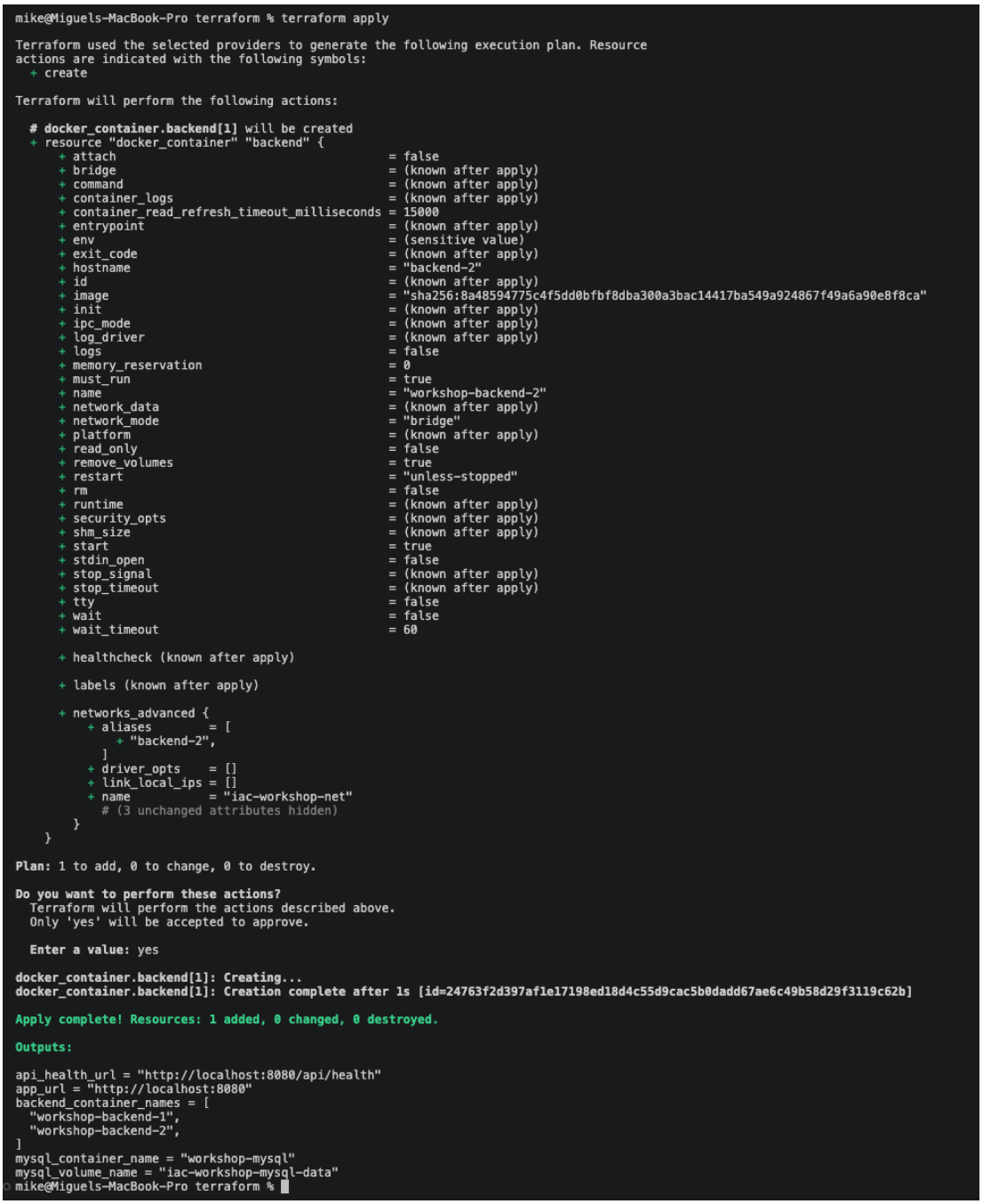

Figura 15: El `apply` recrea únicamente el contenedor faltante.

```
Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

docker_container.backend[1]: Creating...
docker_container.backend[1]: Creation complete after 1s [id=24763f2d397af1e17198ed18d4c55d9cac5b0dadd67ae6c49b58d29f3119c62b]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

api_health_url = "http://localhost:8080/api/health"
app_url = "http://localhost:8080"
backend_container_names = [
  "workshop-backend-1",
  "workshop-backend-2",
]
mysql_container_name = "workshop-mysql"
mysql_volume_name = "iac-workshop-mysql-data"
mike@Miguels-MacBook-Pro terraform %
```

Figura 16: `1 added, 0 changed, 0 destroyed`: el resto del estado seguía coincidiendo con la realidad y no se tocó.

5.5 Reemplazar un recurso puntual

Cuando no hay *drift* pero se quiere forzar una instancia nueva y limpia de un recurso concreto:

```
terraform apply -replace="docker_container.backend[0]"
```

```
mike@Miguels-MacBook-Pro terraform % terraform apply -replace="docker_container.backend[0]"
docker_volume.mysql_data: Refreshing state... [id=iac-workshop-mysql-data]
docker_image.mysql: Refreshing state... [id=sha256:7dcddc01f13bab2f15cde676d44d01f61fc9f99fe7785e86196dfc07d358ae2bmysql:8.0]
docker_image.nginx: Refreshing state... [id=sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10nginx:1.27-alpine]
docker_network.workshop_net: Refreshing state... [id=bbd63b681a36238807005ffbe48ed9a6f3ab23f98553437df1e930da206f3ff7]
docker_image.backend: Refreshing state... [id=sha256:8a48594775c4f5dd0bfbf8dba300a3bac14417ba549a924867f49a6a90e8f8caiac-workshop-backend:local]
docker_image.frontend: Refreshing state... [id=sha256:717da3a9b8ef82d66440ee4186a3f808bbdd028bb568fab7ac385ca42702b5f3iac-workshop-frontend:local]
docker_container.frontend: Refreshing state... [id=736eedc20721f6f29aec40649a070b9e6195ba35646fd6fb43876854f2304e6d]
docker_container.mysql: Refreshing state... [id=81fde5253f988e51fecb0814a065b79d5f2a693ad31c8612955b8726987c468d]
docker_container.backend[0]: Refreshing state... [id=170d7ff57cddea7a2bb8db75229f14b5f2ecbc3ebb8a252211240fcdc4d79d69]
docker_container.backend[1]: Refreshing state... [id=24763f2d397af1e17198ed18d4c55d9cac5b0dadd67ae6c49b58d29f3119c62b]
docker_container.nginx: Refreshing state... [id=e628e258f1d76e17b4043283fffc3338ce698ac75c0982b3457c4d3bc49ed76a]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # docker_container.backend[0] will be replaced, as requested
-/+ resource "docker_container" "backend" {
      + bridge                                      = (known after apply)
      ~ command                                     = [
          - "npm",
          - "start",
        ] -> (known after apply)
      + container_logs                              = (known after apply)
      - cpu_shares                                  = 0 -> null
      - dns                                         = [] -> null
```

Figura 17: El plan marca el recurso como `replaced, as requested` y usa el símbolo `-/+` (destruir y luego crear).

Figura 18: `1 added, 0 changed, 1 destroyed`: solo `backend-1` se recreó; `backend-2` quedó intacto.

Esta bandera sustituye al antiguo `terraform taint`, desaconsejado desde Terraform 0.15.2.

5.6 Renombrar una dirección sin destruir el recurso

Si se renombra un recurso solo en el código, Terraform planea destruir el antiguo y crear uno nuevo, aunque en la práctica sea el mismo contenedor. `state mv` evita esa recreación innecesaria:

```
terraform state mv docker_container.frontend docker_container.web
```

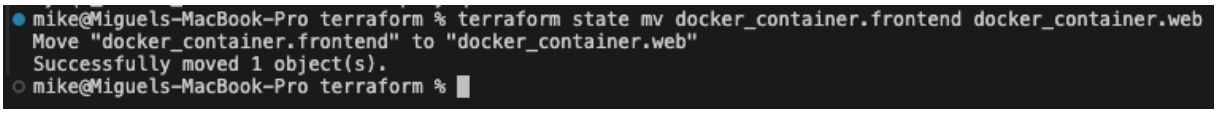

Figura 19: La dirección se mueve dentro del estado: `Successfully moved 1 object(s)`.

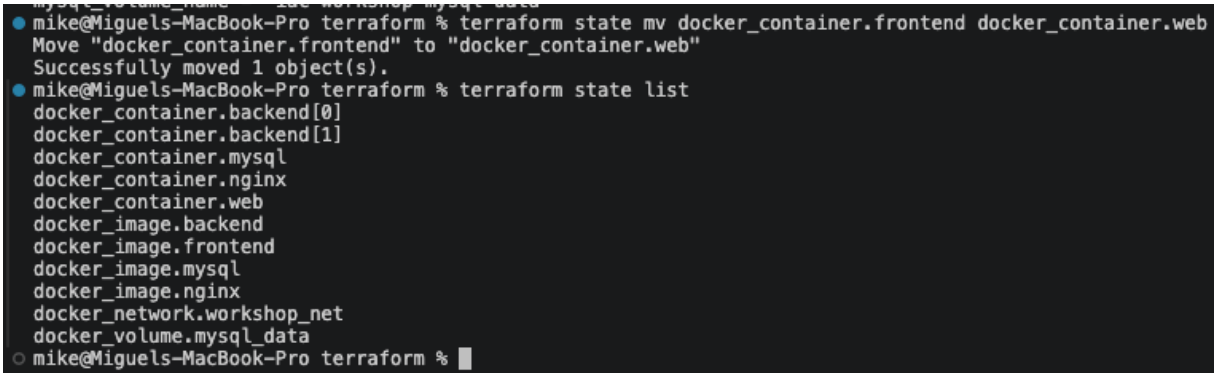

Figura 20: El estado ya contiene `docker_container.web` en lugar de `docker_container.frontend`, sin que el contenedor real se haya tocado.

En este punto el estado y el código quedan momentáneamente desincronizados, y el siguiente `plan` falla. Es un error esperado y didáctico: `nginx.tf` seguía apuntando a la dirección vieja.

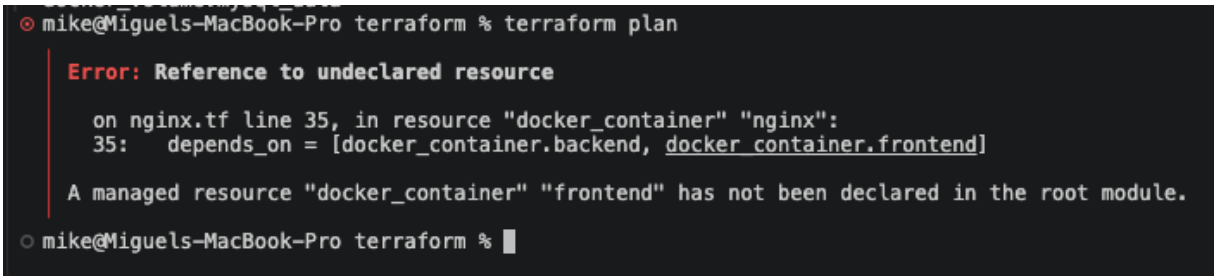

Figura 21: `Reference to undeclared resource`: `nginx.tf` línea 35 todavía referencia `docker_container.frontend`.

Tras renombrar también el bloque en el código, el plan vuelve a mostrar "sin cambios" en lugar de un *destroy/create*, que es exactamente el objetivo del ejercicio.

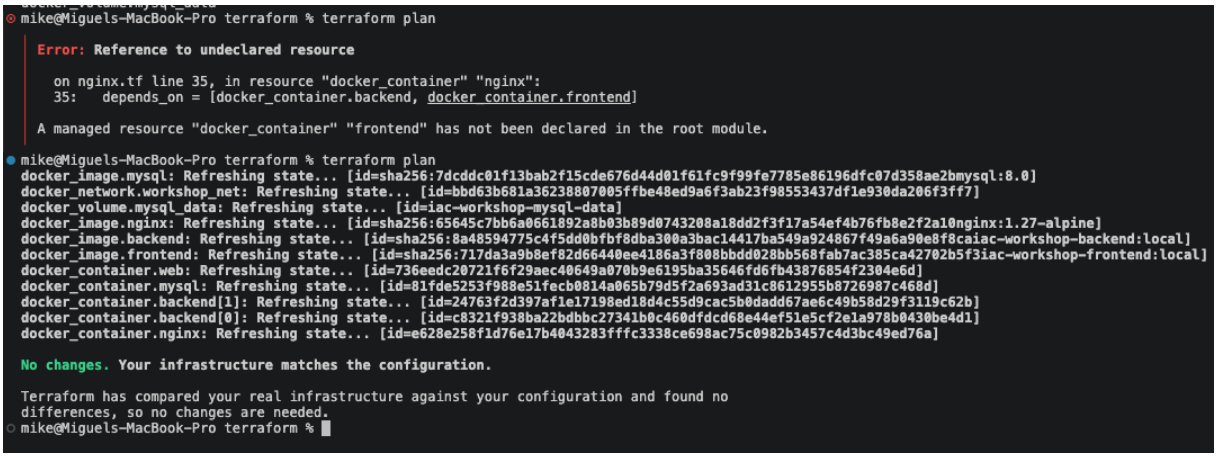

Figura 22: Con código y estado alineados, Terraform no propone ninguna acción.

5.7 Importar un recurso creado manualmente

Este es probablemente el ejercicio más ilustrativo. Se creó un contenedor de Redis **completamente fuera de Terraform**, como si fuera un recurso heredado:

```
docker network connect iac-workshop-net --alias redis-manual \
  $(docker run -d --name manual-redis redis:7-alpine)
```

Figura 23: Docker descarga `redis:7-alpine` y crea el contenedor `manual-redis` al margen de Terraform.

Después se escribió en `terraform/redis.tf` el bloque que *debería* describir ese contenedor, y se adoptó en el estado sin recrearlo:

```
terraform import docker_container.redis \
  $(docker inspect -f '{{.Id}}' manual-redis)
```

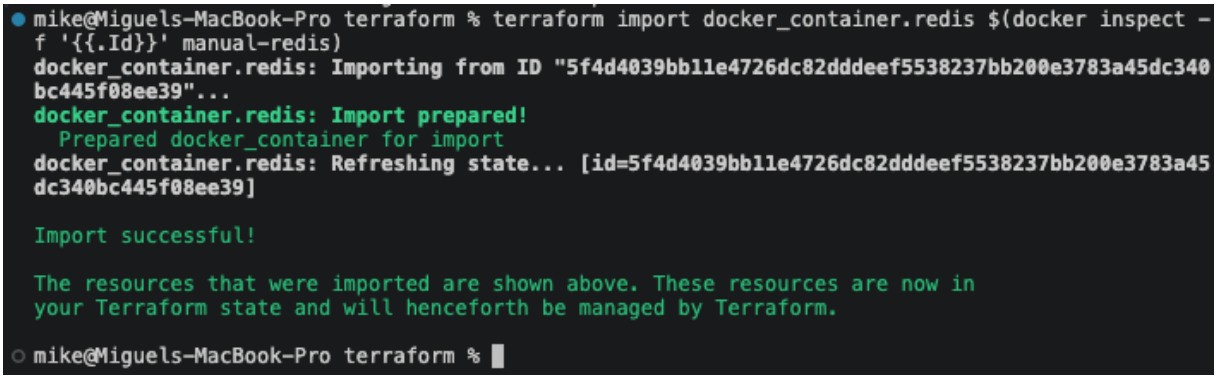

Figura 24: `Import successful!` El contenedor pasa a estar administrado por Terraform sin haber sido destruido ni vuelto a crear.

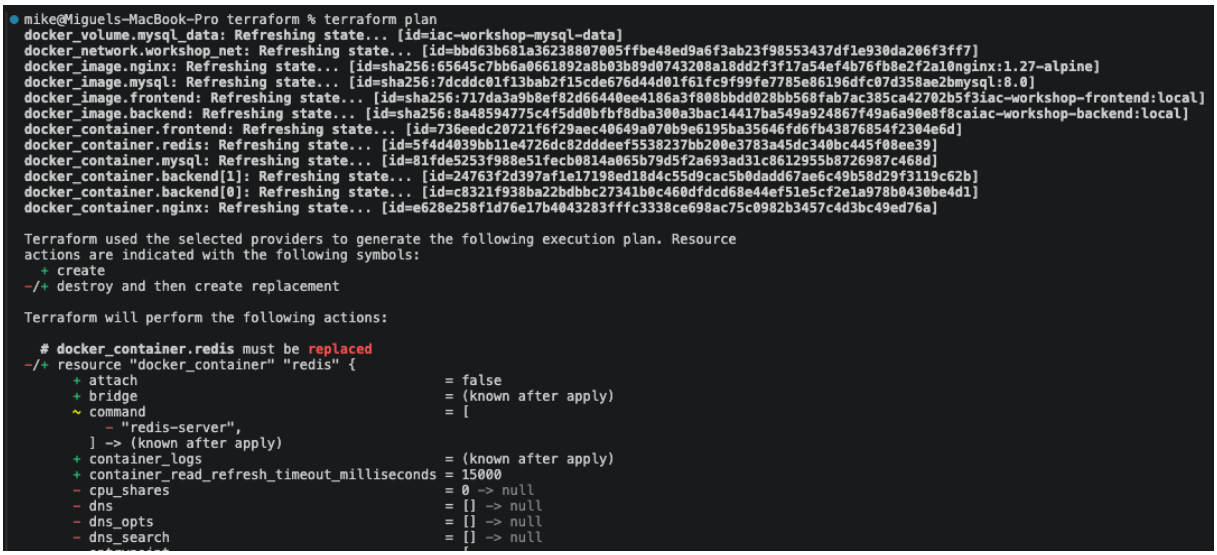

Figura 25: El plan posterior muestra `docker_container.redis must be replaced`: algunos atributos del código no coinciden exactamente con los del contenedor importado (por ejemplo el `command`), lo que fuerza una recreación. Es un resultado normal del ejercicio.

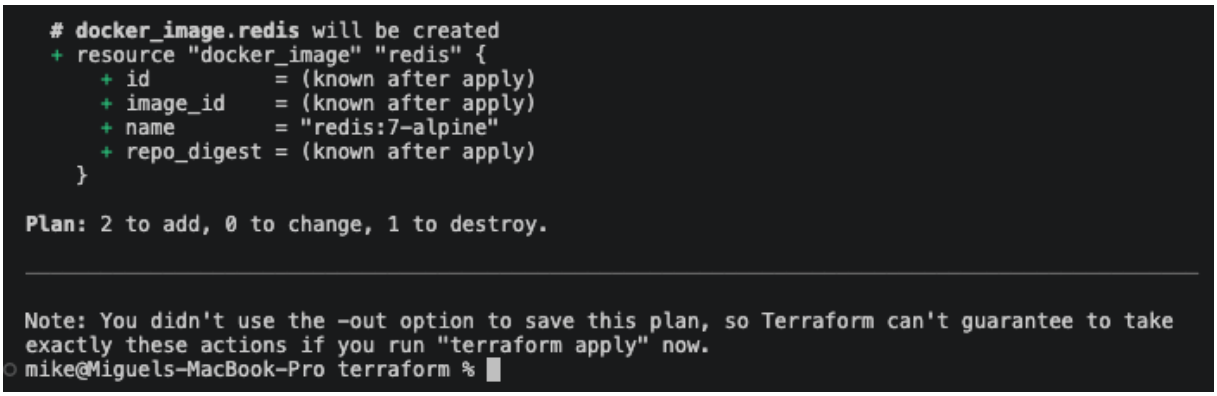

Figura 26: `2 to add, 0 to change, 1 to destroy`: se crea además `docker_image.redis`, porque el recurso de imagen no soporta `import` en este proveedor. Como la imagen ya existía localmente, es un efecto inofensivo.

Figura 27: El `apply` completa la adopción del recurso.

Docker Compose no tiene ningún comando equivalente a `terraform import`: solo puede gestionar lo que él mismo creó. El ejercicio deja además una lección extra: no todos los tipos de recurso de un proveedor soportan importación, y conviene revisar la documentación antes de asumirlo.

5.8 Limpieza del ejercicio y una observación

```
rm terraform/redis.tf
docker rm -f manual-redis
terraform state rm docker_container.redis docker_image.redis
```

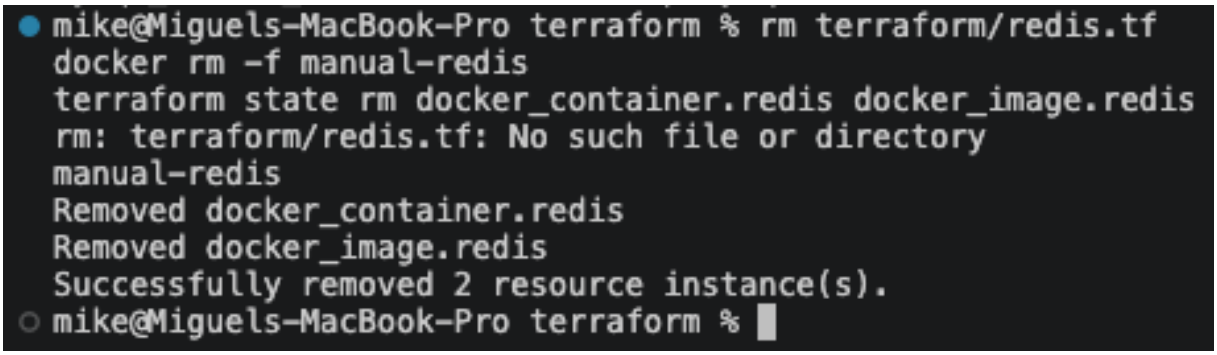

Figura 28: `terraform state rm` saca los dos recursos del estado sin destruirlos en la realidad. Nótese, sin embargo, el mensaje `rm: terraform/redis.tf: No such file or directory`.

Merece la pena detenerse en ese detalle. El comando se ejecutó desde dentro de `terraform/`, así que la ruta relativa `terraform/redis.tf` no resolvía y el fichero **no se borró**. El resultado fue que, en el siguiente `apply` (durante el escalado de la sección siguiente), Terraform volvió a crear Redis a partir de ese `redis.tf` superviviente.

Lejos de ser solo un descuido, es una demostración práctica del principio de fondo del taller: **el código es la fuente de verdad**. Sacar algo del estado no lo elimina de la configuración, y mientras el bloque siga declarado Terraform lo reconstruirá. La ruta correcta desde `terraform/` habría sido simplemente `rm redis.tf`.

5.9 Resumen de la comparación con Docker Compose

Tabla 3: Qué puede hacer cada herramienta

Capacidad	Compose	Terraform
Detectar si la configuración cambió	Sí	Sí
Detectar *drift* por cambios manuales	No	Sí (`plan`)
Vista previa con *diff* explícito	No	Sí (`plan`)
Consultar atributos completos de un recurso	Limitado	Sí (`state show`)
Renombrar sin recrear el recurso real	No	Sí (`state mv`)
Adoptar un recurso creado por fuera	No	Sí (`import`)
Reemplazar un recurso puntual bajo demanda	No	Sí (`-replace`)
Bloqueo contra aplicaciones concurrentes	No	Sí (*backend* remoto)

6. Escalar el backend

Basta cambiar una sola variable en `terraform.tfvars`:

```
backend_replica_count = 4
```

```
terraform plan
```

```
Plan: 5 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  ~ backend_container_names = [
        # (1 unchanged element hidden)
        "workshop-backend-2",
      + "workshop-backend-3",
      + "workshop-backend-4",
    ]

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly
these actions if you run "terraform apply" now.
mike@Miguels-MacBook-Pro terraform %
```

Figura 29: El plan añade `workshop-backend-3` y `workshop-backend-4` a la lista de *outputs*.

Lo interesante no son las réplicas nuevas, sino que Nginx también aparece marcado para recreación. Su configuración se genera con `templatefile()` a partir de la lista de réplicas; al cambiar esa lista cambia el contenido del fichero, y como el fichero se sube al crear el contenedor, no se puede parchear en caliente. Es el principio de infraestructura inmutable, visible en la práctica.

```
  # docker_container.nginx must be replaced
-/+ resource "docker_container" "nginx" {
      + bridge                                  = (known after apply)
      ~ command                                 = [
          - "nginx",
          - "-g",
          - "daemon off;",
        ] -> (known after apply)
      + container_logs                          = (known after apply)
      - cpu_shares                              = 0 -> null
      - dns                                     = [] -> null
      - dns_opts                                = [] -> null
```

Figura 30: `docker_container.nginx must be replaced`: la recreación no viene de un cambio manual, sino de una dependencia sobre datos que cambiaron.

```
terraform apply
```

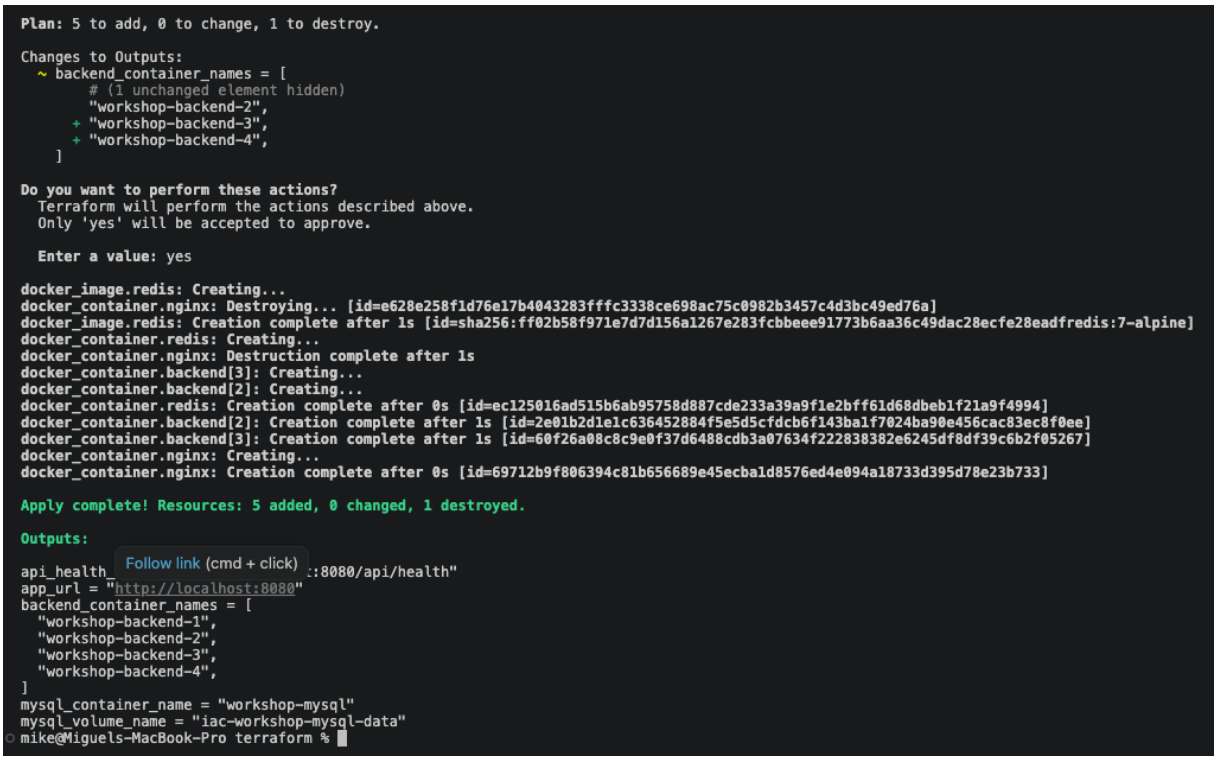

Figura 31: `5 added, 0 changed, 1 destroyed`: dos backends nuevos, Nginx destruido y recreado, y Redis recreado a partir del `redis.tf` que había sobrevivido a la limpieza.

Volviendo al navegador, el botón de balanceo ahora reparte las peticiones entre cuatro hostnames distintos.

Figura 32: La misma interfaz, ahora repartiendo entre `backend-1` a `backend-4`. El contador de visitas subió a 2, leído desde el volumen persistente de MySQL.

7. Destruir el entorno

```
terraform destroy
```

Figura 33: `Destroy complete! Resources: 15 destroyed.` Terraform elimina todo en orden inverso de dependencias: primero los contenedores, luego las imágenes, y al final el volumen y la red.

Los 15 recursos son los 11 originales más los 2 backends añadidos al escalar y los 2 de Redis. Conviene subrayar que `destroy` se llevó también el volumen `iac-workshop-mysql-data` y, con él, los datos de MySQL. En un entorno con datos reales, `lifecycle { prevent_destroy = true }` sobre ese volumen es la red de seguridad contra exactamente este comando.

8. Consideraciones antes de llevarlo a producción

El proveedor `docker` es excelente para aprender y para desarrollo local, pero no está pensado para producción. Los ajustes que un equipo debería hacer antes de dar ese salto:

1. **Cambiar de orquestador.** Este proveedor administra un solo host: no reinicia nada si el host cae, no distribuye carga entre máquinas y no tiene autoescalado. El mismo enfoque declarativo se aplica con Terraform sobre Kubernetes o ECS.
2. **Backend remoto de estado, con bloqueo.** El `terraform.tfstate` local es inviable en equipo: dos personas aplicando a la vez pueden corromperlo.

3. **Nunca variables sensibles con `default`.** Las contraseñas no deberían tener valor por defecto, ni vivir en un `.tfvars` versionado, sino inyectarse desde un gestor de secretos.
4. **Tags de imagen inmutables, nunca `:latest`**, para poder hacer *rollback* con certeza.
5. **Límites de recursos por contenedor** (`memory`, `cpu_shares`).
6. **TLS en el punto de entrada**, terminando en el balanceador.
7. **Backups del dato**, o mejor, una base de datos gestionada.
8. **`prevent_destroy`** en los recursos con estado crítico.
9. **Pipeline de CI/CD con aprobación humana**, políticas como código y escaneo de imágenes.
10. **Usuario no root en los contenedores** (ya se cumple en el backend, que usa `USER node`).
11. **Monitoreo y logging centralizados.**
12. **Ambientes separados con estados completamente distintos**, para limitar el radio de explosión de un error.

9. Referencia rápida de comandos

Tabla 4: Comandos utilizados en el taller

Comando	Qué hace
`terraform init`	Descarga proveedores y prepara el directorio
`terraform fmt`	Normaliza el formato del código
`terraform validate`	Revisa sintaxis y referencias, sin tocar Docker
`terraform plan`	Calcula y muestra el plan, sin aplicar nada
`terraform apply`	Ejecuta el plan (pide confirmación)
`terraform destroy`	Elimina todos los recursos administrados
`terraform state list`	Lista las direcciones del estado
`terraform state show`	Muestra los atributos de un recurso
`terraform show`	Vuelca el estado completo en formato legible
`terraform state mv`	Renombra una dirección sin destruir el recurso
`terraform state rm`	Deja de administrar un recurso, sin destruirlo
`terraform import`	Adopta un recurso ya existente
`terraform apply -replace`	Fuerza la recreación de un recurso
`terraform output`	Muestra los valores de salida

10. Conclusiones

- El flujo `init` → `plan` → `apply` se completó sin incidencias: 11 recursos creados y la aplicación respondiendo en `localhost:8080`, con el balanceo entre réplicas verificable desde el navegador.
- El archivo de estado es la diferencia real frente a Docker Compose. Detectar el borrado manual de un contenedor, renombrar una dirección sin recrear nada y adoptar un contenedor creado a mano son operaciones que Compose sencillamente no puede ofrecer, porque no mantiene un registro consultable de la infraestructura.

- El escalado dejó ver el efecto en cascada más interesante del taller: cambiar una variable no solo añadió réplicas, sino que obligó a recrear Nginx, porque su configuración se deriva de esa misma variable. Terraform razona sobre el grafo de dependencias, no sobre ficheros sueltos.
- El descuido en la limpieza de Redis resultó ser el ejemplo más claro de todo el ejercicio: sacar un recurso del estado no lo borra del código, y mientras el bloque siga declarado Terraform lo reconstruirá en el siguiente `apply`.
- El `destroy` final eliminó los 15 recursos, incluido el volumen de MySQL, confirmando por qué `prevent_destroy` es necesario en cuanto hay datos que importan.